AI Mathematician — Phase 1 Report

Search · ATP (Vampire) · ITP (Lean 4) · LLM proposer, evaluated on the 999-theorem Megalodon reference library. October 2026.

1. Summary

- **877 / 999 theorems (87.8%) proved**, each held out and proved only from facts that precede it in the library.
- **665** without any LLM (640 in the v4 pipeline run, 25 more from induction and earlier runs), and **212** by the LLM loop, all on theorems the pipeline had attempted and failed.
- Every proof is checked by Lean 4's kernel, with the proof's statement compared to the original as elaborated terms and a leak guard rejecting any use of the target or later facts. All 640 v4 proofs were re-verified in fresh Lean processes (640 / 640), and every LLM proof was re-verified from scratch (212 / 212).
- Vampire and the LLM only propose proofs; every proof is verified by Lean.
- The decisive finding for the LLM: its failures were mostly about missing lemmas, not reasoning. Showing it 96 to 160 facts instead of 24 added about 100 proofs (Section 6.3).
- Progression: 418 → 514 → 563 → 640 without an LLM (v1–v4), then 742 → 755 → 832 → 859 → 877 with the LLM loop (v5–v9).

2. Phase 1 scope and status

Phase 1 of the design sketch: an LLM proves a lemma and formalizes the proof, guiding an ITP that checks it with the help of an ATP, fed by Search + Download. Mapping to what is built:

Sketch component	Implementation	Status
Search + Download	Premise selection over the Megalodon library (symbol overlap, naive Bayes learned from earlier proofs, k-NN, recency)	Done; Download = retrieval from the library (external sources: later)
ATP	Vampire 5.1.0, higher-order TPTP (THF)	Done
ITP	Lean 4.34.0 over the verified Megalodon→Lean 4 translation (999/999 theorems)	Done

Sketch component	Implementation	Status
LLM guiding the ITP	LLM writes Lean tactic scripts; Lean's errors fed back for up to 7 rounds	Done: 212 theorems (Section 6)
LLM proving lemma → formalizing	Draft–Sketch–Prove: informal proof → Lean outline with gaps → ATP/ITP tactics fill gaps	Done, evaluated (Section 6)

3. Evaluation setting

Library. Megalodon's `100thms_12.mg` (1,158 declarations: 999 theorems, 139 definitions, 13 axioms, 6 primitives, 1 parameter), exported from Megalodon's checked proof terms and translated to Lean 4. The translation is verified: all 999 original proofs compile, using only Megalodon's 17 foundational axioms. No Mathlib; Megalodon's own higher-order set theory with Church-encoded logic.

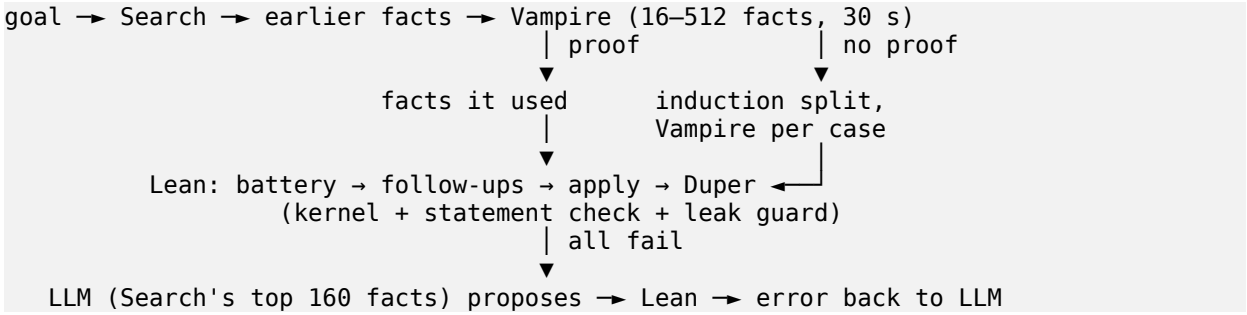
Held-out protocol. Each theorem is attempted in turn with its proof hidden. The prover may use only declarations with a smaller index (earlier in the library). Search, premise-selection training and induction principles are all restricted accordingly.

Acceptance. A proof counts only if all of the following hold:

1. Lean's kernel accepts it.
2. Its statement is identical to the original theorem's, compared as elaborated Lean expressions (not text).
3. The leak guard passes: the transitive closure of constants used by the proof term (through auxiliary definitions, with theorem bodies unfolded) contains no library constant at or after the target's index.
4. No `sorryAx`.

The compiled library contains all 999 theorems, so without (3) a proof could cite its own answer or a later equivalent. The guard is tested both ways: citing the target, or a later theorem with an identical statement, must be rejected; a genuine proof, or citing an earlier identical theorem, must be accepted. The first guard version passed cheating proofs (Lean hides theorem bodies unless asked with `allowOpaque`); these tests caught it.

4. Pipeline



4.1 Search (premise selection)

Ranks the facts preceding the goal by a sum of: IDF-weighted symbol overlap (MePo-style); k-NN votes from the dependencies of the most similar earlier theorems; a recency term; and a naive Bayes score in the style of MaSh, trained on which facts earlier proofs used, over statement features that include shape (e.g. conclusion head, head–argument pairs, hypothesis heads), not only symbols. Name similarity is deliberately off: a newly conjectured lemma has no descriptive human name, so using it would inflate the benchmark.

Recall: all facts the original proof cited are in the top k	**old @32**	**new @32**	**old @64**	**new @64**
Second half of library (not used for tuning)	31.9%	36.2%	46.4%	51.8%
327 non-induction theorems Vampire failed on (v1)	12.2%	19.6%	22.6%	33.0%

4.2 ATP: Vampire

Goal and ranked facts are rendered as THF; Megalodon's encoded connectives (`and`, `or`, `iff`, `ex`, Leibniz `eq`, …) map to native ones, and definitions are included transitively (depth 3). Vampire runs in portfolio mode for 30 s on the top 16, 32, 64, 128, 256 and 512 facts (v2 used 5 s on 16/32/64; v4 added 256 and 512). Its proof is not reused directly: the facts it used become the premise hints for Lean. Vampire proofs are not unique, and the facts of one proof may suit Lean better than another's, so every distinct premise set is kept.

4.3 ITP: Lean reconstruction

Kernel-checked bridge lemmas (`b_and`, `b_or`, `b_ex`, `b_eq`, …) turn the encoded logic into Lean's native logic so `grind` applies. A deterministic battery of tactic scripts is built from the hints: intro/unfold then `grind`; premises introduced as hypotheses then `grind`; `solve_by_elim`; unfolding only the goal's own definitions; set extensionality. The last two

were distilled from the proofs the LLM found in v1. If the whole battery fails, a slower second line runs (v3): premises as `grind` e-matching lemmas, higher `grind` effort, and `simp_all` with premises and definitions. v4 added a third line of `apply` strategies and a fourth, Duper, a superposition prover running inside Lean that handles the higher-order instances `grind` cannot.

Strategy (first success)	Theorems
prem-grind	302
prem-unfold-grind	133
unfold-grind	50
solve_by_elim	40
duper	35
intro-grind	33
induction	14
duper-unfold	9
ext-grind	8
simp_all-prem	3
apply-2	2
goal-unfold-sbe	2
goal-unfold-grind	2
apply-0	2
apply-1	2
unfold-grind-big	1
apply-unfold-3	1
grind-lemmas	1

4.4 Automatic induction

Vampire does not invent induction predicates: given exactly the facts the original proof cited, it proved only 2 of 40 sampled induction theorems. When Vampire finds no proof, the pipeline reads the goal as P(x) for each variable x with a matching guard, instantiates each available principle (nat_ind, nat_complete_ind, ordinal_ind, SNoLev_ind, finite_ind, In_ind; only those proved before the goal), and sends each case to Vampire separately. In Lean: introduce, revert all but x and its guard, `apply` the principle (the motive is found by higher-order pattern unification), and prove each case with the battery. Each case is first tried as its own declaration, since Lean's heartbeat budget is per declaration.

In the full run, induction proved 14 theorems (nat_ind 13, finite_ind 1), including commutativity of addition and of multiplication and associativity of addition on the naturals.

5. Results without an LLM

Run	Proved	Rate
v1: Search (overlap, k-NN, recency) + Vampire + battery	418	41.8%
v2: + learned Search, induction, distilled strategies, all Vampire proofs	514	51.5%
v3: + Vampire 30 s with a 128-fact slice, second-line Lean strategies	563	56.4%
v4: + Duper, 256/512-fact slices, `apply` strategies	**640**	**64.1%**
v4 + induction and earlier runs (no LLM)	665	66.6%

Category	Theorems	v1	v3	v4	Final (with LLM)
Propositional logic	39	36 (92%)	38 (97%)	38 (97%)	39 (100%)
Set theory	179	96 (54%)	123 (69%)	133 (74%)	177 (99%)
Natural numbers	119	48 (40%)	81 (68%)	87 (73%)	115 (97%)
Ordinals	56	31 (55%)	37 (66%)	39 (70%)	51 (91%)
Surreal numbers	606	207 (34%)	284 (47%)	343 (57%)	495 (82%)

Categories from the translator's vocabulary heuristic; approximate.

v2 against v1: 113 theorems gained, 17 lost. v3 against v2: 57 gained, 8 lost; v4 against v3: 82 gained, 5 lost. The losses are borderline goals where Vampire found a different proof, or none within its time limit; the pipeline has run-to-run variation on such goals.

Unproved by v4: 359. For 59 of them Vampire found a proof that the Lean battery could not rebuild; for 300 Vampire found none.

How v3 was chosen. Two targeted experiments on the v2 failures, before the v3 run: (1) on the 395 goals with no Vampire proof, Vampire with 30 s and a 128-fact slice found 66 proofs (most only at 128 facts), of which Lean rebuilt 32; a larger budget for the induction cases helped little (1 of 42 sampled). (2) On the 90 goals with a Vampire proof but no reconstruction, the second-line strategies rebuilt 6. Both were then built into the pipeline and measured by the single v3 run above.

The typical remaining reconstruction failure needs a higher-order instance of a premise (e.g. the choice axiom `Eps_i_ax` at a specific predicate), which Vampire finds and `grind` does not.

Time per goal (one goal at a time, as in use): in v4, proved goals take a median of 60 s, 90% within 266 s; unproved goals are abandoned after a median of 265 s.

6. The LLM loop

The LLM sees the goal, Lean's goal state, the definitions used and the selected earlier facts, never the original proof or anything after the goal. Everything it writes goes through the same acceptance checks. Models are served through an OpenAI-compatible endpoint.

6.1 First evaluation: direct Lean with error feedback (v1)

Target: the 140 theorems where Vampire found a proof but the v1 battery could not rebuild it, with the same Vampire premise sets for every model. Up to 3 rounds, Lean's error returned each round.

Model (28-theorem sample)	Proved	Avg time / theorem
deepseek/deepseek-v4-flash	11 / 28	127 s
openai/gpt-oss-120b	9 / 28	46 s
minimax/minimax-m3	7 / 28	496 s (10 API timeouts)
qwen/qwen3.8-27b	2 / 28	550 s (26 API timeouts; inconclusive)

Cascade on all 140: gpt-oss-120b proved 41, then deepseek-v4-flash 10 of the remaining 99: **51 / 140 (36%)**, all re-verified in fresh Lean runs. 30 of these 51 are now proved by v3 without an LLM, because the strategies distilled from them generalise.

6.2 Draft, sketch, prove (v3)

The sketch's "LLM proving lemma → LLM formalizing proof" path, in the style of Draft, Sketch, and Prove (Jiang et al., ICLR 2023):

1. **Draft** (1 call): the model writes an informal numbered proof, naming the facts each step uses and, for induction, the principle and predicate.
2. **Sketch** (1 call): it formalizes that proof as a Lean outline of intermediate `have` steps, leaving any routine step as `sorry`.
3. **Prove** (no LLM): each `sorry` is replaced by the automatic tactics (`grind`, `solve_by_elim` over the selected facts); Lean checks the whole.
4. **Repair** (1 call): on failure, Lean's error goes back once; the model revises the outline. A script rejected only because it continues after the goal is closed is retried with trailing lines removed (no call).

Budgeted run. Hard cap of 100 calls on gpt-oss-120b. Sample: every 15th of the 485 theorems the v2 pipeline did not prove (33 theorems, not hand-picked). The first was a live test; it failed on a script that continued after the proof was complete, which motivated

the trimming step. Its stored reply passes after trimming but is **not counted**. The remaining 32 were run with the final code.

Result: 0 / 32 proved (0.0%) under the same acceptance checks as every other proof. API usage for the whole pass: 99 calls, 438,254 prompt and 287,695 completion tokens.

Final failure reason	Theorems
Lean heartbeat budget exhausted while filling gaps	13
Encoded logic / opaque sets treated as Lean-native	11
Type mismatch	3
Wrong number of intro steps	2
Other	3

Post-hoc check (no API calls). All 64 stored outlines were re-checked with 5× the heartbeat budget: 0 / 64 pass, and the same 20 time out at the same points. The gaps the model leaves are too coarse for the automatic tactics, rather than cut short by the budget.

Reading. The path works end to end, but on this hard remainder (only 4 of the 32 had a Vampire proof) a general-purpose model with 3 calls does not produce usable outlines in Megalodon's encoding. The clearest levers: few-shot examples drawn from the 51 successful direct-mode proofs and the 563 pipeline proofs, finer outlines (more, smaller `have` steps), and per-gap checking so that each gap is reported back to the model individually.

6.3 The fixed LLM loop (v5–v9)

From v5 the LLM loop runs over every theorem the pipeline leaves unproved: up to 7 rounds per theorem, each round a Lean check whose error goes back to the model. Model: deepseek/deepseek-v4-flash, served through the SingularityNET gateway.

Version	What changed	Proved	Gain
v4 + earlier	No LLM	665	—
v5	Hidden failures fixed (73% of replies were empty: the model used the gateway's 8,192-token reply limit on thinking); free repairs (indentation, `eq` bridge, finishing leftover goals); few-shot examples	742	+77
v6	Real feedback: in 49% of failed attempts the model had been shown a misleading indentation error; scripts are normalized before checking	755	+13
v7	96 facts per theorem instead of 24	832	+77
v8	160 facts, 7 rounds	859	+27

Version	What changed	Proved	Gain
v9	Lemmas shown as Lean prints them; a misapplied lemma is answered with its signature	**877**	+18

Missing lemmas, not reasoning. On the 257 hardest theorems the human proofs cite a median of 13 earlier lemmas. Search's top 24 facts contained all of them for only 7 theorems; the top 96 for 66 (recall 0.45 at 24, 0.79 at 96, 0.87 at 160). Raising the facts shown from 24 to 96 added 77 proofs in one run, many in the first rounds, including theorems that hours of decomposition had failed on.

What did not help. Showing the model Vampire's whole proof (34 / 51 with it, 34 / 51 without, same facts); planning a proof as an outline whose steps are proved as separate lemmas (most steps proved, a hard core step stayed open: 0 / 3 theorems); embedding-based fact retrieval (recall 0.48 at 96 facts against Search's 0.79); and other models on the hardest theorems: gpt-oss-120b, gemma-4-31b, minimax-m3, asi1 and qwen3.8 proved none of the goals tried, including goals deepseek-v4-flash proves.

Cost and stopping. The LLM work used about 11,800 calls (162 M input and 59 M output tokens). New proofs cost about 26 calls each in v7 and about 74 in v9, as the remaining theorems grow harder; Phase 1 stops at 877, with 122 theorems unproved, mostly long surreal-number and integer-arithmetic proofs.

7. Limitations

- One library (999 theorems) in one foundation; results may not transfer unchanged to other libraries or to Mathlib-style Lean.
- Download is retrieval from this library only; no external sources yet.
- Run-to-run variation on borderline goals (Vampire's time limit, LLM sampling): for example, 5 v3 successes were not reproduced in v4.
- Category figures use a vocabulary heuristic.
- The LLM results depend on one model (deepseek-v4-flash) on one gateway, whose response time was the main bottleneck.
- Speed was not optimised: each Lean check starts a fresh process and reloads the library, and a batch finishes every candidate even after one succeeds.

8. Next steps

1. Phase 1 is closed at 877 / 999.
2. Phase 2: Hyperon drives Lean. A MeTTa inference controller decides each proof step (automation, a lemma to apply, an LLM suggestion, a split) and Lean checks it;

the library and the proved theorems move into a Math Atomspace, so lemmas are retrieved from memory rather than re-proved. Step-by-step Lean is feasible: through the Lean REPL the library loads in 0.2 s and each step takes under 0.01 s.

3. Phase 3: PLN feeding the inference controller.

Appendix: reproducing

```
cd phase1
python3 test_guard.py                                  # leak-guard controls
python3 experiment.py 1 6 ../results/phase1_full_v4.jsonl   # full run, no LLM
python3 recheck_full.py ../results/phase1_full_v4.jsonl 6 ../results/recheck_v4.jsonl
# LLM loop over every theorem still unproved, then re-verification
PYTHONPATH=. python3 experiments/try_llm_vampire_sweep.py prep-search
../results/rest.jsonl --facts 160
PYTHONPATH=. python3 experiments/try_llm_vampire_sweep.py run
../results/rest.jsonl ../results/out.jsonl noproof \
    --rounds 7 --workers 4 --examples 3 --model deepseek/deepseek-v4-flash-0731
PYTHONPATH=. python3 experiments/verify_vp_final.py ../results/verified.json
../results/out.jsonl
```

Repositories: ai-mathematician (this system) and megalodon-lean4 (the verified Megalodon → Lean 4 translator).